

Database Design Case Study

Critical evaluation of 3-Tier Architecture implementing
Relational and NoSQL Databases

University Of Gloucestershire
CT6049 [REDACTED]
21st January 2026

Contents

1	3-Tier Architecture	2
1.1	Overview	2
1.2	Presentation Tier	2
1.2.1	Implementation	3
1.3	Data Tier	7
1.3.1	Implementation	8
1.4	Application Tier	10
1.4.1	Implementation	14
2	Database Modelling Evaluation	18
2.1	Relational (PostgreSQL)	18
2.1.1	Normalization	19
2.2	NoSQL (MongoDB)	20
2.2.1	Usage in Implementation	21
	Appendices	23
.1	Additional Figures	23
A	Backend Implementation	25
A.1	Configuration & Entry Point	25
A.2	Controllers	27
A.3	Services	34
A.4	Domain Models & DTOs	44
A.5	Security Configuration	50
B	Data Persistence Layer	56
B.1	Repository Interfaces	56
B.2	SQL Implementation	58
B.2.1	Adapters	58

B.2.2	Entities	62
B.2.3	Internal Repositories	68
B.3	MongoDB Implementation	69
B.3.1	Adapters	69
B.3.2	Documents	73
B.3.3	Internal Repositories	77
B.4	Database Seeders	78
C	Frontend Implementation	89
C.1	Core Application	89
C.2	Components	94

List of Figures

1.1	Frontend File Structure	4
1.2	api.js Request Function	5
1.3	Dashboard ToF	7
1.4	Generic LoanRepository.java	8
1.5	Extending Generic Interface	10
1.6	Loan Domain Entity	12
1.7	Loan DTO	13
1.8	LoanController Snippet	15
1.9	LoanService Snippet	17
2.1	Simplified ERD	20
2.2	mongoLoanDoc Fields	22
3	Whole Application Structure	24

Chapter 1

3-Tier Architecture

1.1 Overview

In software development, a programs architecture refers to the logical flow and manipulation of data; therefore, by its very name, 3-Tier is a "modular client-server architecture that consists of a presentation tier, an application tier, and a data tier." (Gillis 2024).

This architectural design can be seen practiced in all major modern applications due to its complete **Separation of Concerns**. Each of the 3 layers receive input from another layer, performs its actions, and then sends that data onto the next layer, without concern for what happens to the data before and after that layers effects.

By decoupling the presentation of the data, from the logic of the program, and lastly from the storage of the data, systems become more maintainable, scalable and secure.

1.2 Presentation Tier

The presentation tier, is typically the last "stop" for all data traversing an application, this is the visible parts of an application; the UI, the frontend, etc. Users interact with the application through this layer, requesting information to travel "up" the chain, or pushing data down. This could be present in the form of a web interface, a desktop application, or even direct web requests through tools like `curl`.

Often referred to as the "dumb" layer, the presentation tier essentially receives all

the information it needs from the application tier through API requests and displays the received data in a presentable manor. While in more complex systems, this layer might be a whole separate application, and may include logic relating to the translation of said data, it still fundamentally takes important data in, makes the data presentable, and then displays it to the user.

Additionally, this layer also has to interpret user input, however crucially even user input is not acted upon here; instead opting to delegate this task to the application tier. This further keeps the layers concerns separated, but more importantly bolsters security.

1.2.1 Implementation

To stay true to the above description, my database application uses a Web interface that runs completely separately to any other layers. fig. 1.1, and more generally fig. 3, show the structure of this application. Due to familiarity, Svelte was chosen for this frontend, while not as popular as others, Svelte is just a component framework and does not perform any server-side logic itself.

As such, this frontend needs a way to communicate to the backend service; fig. 1.2 is a helper function that was created to facilitate this, with the full implementation of this API file shown in appendix C.1.



Figure 1.1: Frontend File Structure

```

import { authToken } from "../stores";

const API_BASE = 'http://localhost:8082';
let token;

authToken.subscribe(value => {
  token = value;
})

async function request(path, opts = {}) {
  const url = API_BASE + path;
  const headers = opts.headers ? { ...opts.headers } : {};

  if (token) headers['Authorization'] = 'Bearer ' + token;
  headers['Content-Type'] = headers['Content-Type'] || 'application/json';

  const res = await fetch(url, { ...opts, headers });
  if (res.status === 401) {
    authToken.set(null);
    throw new Error("Unauthorized");
  }

  let data;
  const contentType = res.headers.get('content-type') || '';
  if (contentType.includes('application/json')) {
    data = await res.json();
  } else {
    data = await res.text();
  }

  if (!res.ok) {
    const err = (typeof data === 'object' && data.error) ? data.error : (data || res.statusText);
    throw new Error(err);
  }

  return data;
}

```

Figure 1.2: api.js Request Function

As this above function is the only means of communication between server and client, the frontend is bound to a strict requirement to follow 3-Tier. For instance, the dashboard component (appendix C.2) acts purely as a consumer of data gathered through a host of API calling functions present at the top of the file: fig. 1.3.

While, in theory the frontend receives enough information to determine the status of each loan, and whether to strike the user with a fine; in practice the application layer performs these calculations, and passes `loan.status` elements that the frontend acts upon. (bottom of fig. 1.3)

This immediately demonstrates an important benefit to this design philosophy; as the logic for status and fineAmounts exist entirely on the backend server behind API requests, a user cannot manipulate the data to suit them. Trying to bypass penalties or falsify data is non-persistent, the User Interface merely reflects the data passed in from the actual deciding forces.

```

    async function handlePay(loanId, fineAmount) {
      if (!confirm(`Are you sure?\nPayment Due: £${fineAmount}`)) return;
      try {
        await payFine(loanId);
        addAlert(`Payment of £${fineAmount} complete!`, "success");
        await load();
      } catch (err) {
        addAlert(err.message);
      }
    }

    async function loanReturn(loanId) {
      try {
        await returnBook(loanId);
        addAlert(`Book returned`, "success");
        await load();
      } catch (err) {
        addAlert(err.message);
      }
    }

    async function load() {
      try {
        loans = await fetchLoansForUser();
      } catch (err) {
        addAlert(err.message);
        if (err.message === "Unauthorized") {
          push("/login");
        }
      } finally {
        loading = false;
      }
    }

    onMount(() => {
      load();
    });

    $: currentLoans = loans.filter((loan) => !loan.returnDate);
    $: finesList = loans.filter((loan) => loan.status === "OVERDUE");

```

Figure 1.3: Dashboard ToF

1.3 Data Tier

On the complete opposite end of the architecture sits the Data Tier. A layer dedicated to the persistence and retrieval of stored data, and the managing of database specific constraints. Just like the presentation layer, no logic fundamental to the performance of the application is present here, and in many applications this layer only consists of the operating database and functions to save and load data.

This layer exists as essentially a translator between storage and logic, and acts like a wall for the application layer. With a proper data tier setup, the application layer will not be concerned with the storage format used by databases. When the application layer needs some data, it should call a function provided by this layer and receive back the data requested, irregardless of if that data was stored in a SQL

databases, in a flat file, or even externally on magnetic tape. This separation opens up a host of available storage methods.

For example, every application has required a database change at some time or another, potentially due to maintenance; in a 3-Tier application, databases can be swapped in and out like modules rapidly and without any interruptions to key business logic.

Or potentially an application wants to store data within two databases synchronously, while only pulling from one. To the core application, nothing changes for a system like this. The same `save` and `load` functions are called.

1.3.1 Implementation

To achieve what's described above a common implementation is the Repository Design Pattern. A set of abstract functions that handle all databases operations for the program on their behalf. GeeksforGeeks 2025 give a good analogy, linking this pattern to a library: "Imagine you're at a library...you don't go into the storage room to search for it yourself, instead you ask the librarian... [they] know where the books are kept and can give you the book you want without having to worry about where it's stored."

In my implementation this takes shape through generic interfaces `LoanRepository`, `BookRepository`, etc. These repository interfaces define the "contract" between data objects and the application layer, resulting in deterministic operations. One of these repository interfaces is shown in fig. 1.4 with others present in appendix B.1

```
public interface LoanRepository {
    Loan save(Loan loan);

    Optional<Loan> findById(String loanId);

    List<Loan> findByUserId(String userId);

    List<Loan> loanHistoryBetweenDates(String userId, LocalDate beginDate, LocalDate endDate);

    boolean doesUserHaveActiveLoanForBookId(String userId, String bookId);
}
```

Figure 1.4: Generic `LoanRepository.java`

The requirements for this application also called for both, SQL and MongoDB databases to be present within the same code base; having these generic repositories allowed for further abstraction. Extending these interfaces, the detailed implementations could be created; one for each specific use case. fig. 1.5 shows snippets of these "Adapter" classes extending the generic interfaces and overriding the defined methods adding logic.

- **Profile Annotations:** Present in both Adapter classes the "Profile" annotation can be seen. My application's backend was built using the spring(boot) framework, so through convince the **Spring Profiles** component was used to facilitate the switching of the connected database. While typically used to configure applications based on the stage of development they are in, I've used profiles to determine what implementations are injected into the application.

While uncommented, the top lines in appendix A.1, can be used to designate what profile is active (sql or mongo). The script used to "launch" the whole application also sets the active profile through a JVM flag:

```
spring-boot.run.profiles=$PROFILE
```

- **Implementation Differences:** These figures also make clear the structural differences between the two databases and why these adapters are necessary. In fig. 1.5a, the **save** method's code is noticeably larger than that in fig. 1.5b. This is because the SQL implementation must adhere to Referential Integrity, requiring the corresponding Entity objects to first be recalled and referenced before the loan can be saved.

In contrast, the Mongo database treats loans as a self-contained documents and stores user and book IDs as raw strings. This demonstrates how even though each database type have their own constraints, following the correct adapter interface pattern abstracts these challenges away from the application logic, letting the core program deal with just the simplified Loan objects within its own domain.

```

@Repository
@Profile("sql")
public class sqlLoanAdapter implements LoanRepository {

    private final JpaLoanRepo jpaRepo;

    private final JpaUserRepo userRepo;
    private final JpaBookRepo bookRepo;

    public sqlLoanAdapter(JpaLoanRepo loanRepo,
        JpaUserRepo userRepo,
        JpaBookRepo bookRepo) {
        this.jpaRepo = loanRepo;
        this.userRepo = userRepo;
        this.bookRepo = bookRepo;
    }

    @Override
    public Loan save(Loan loan) {
        sqlLoanEntity entity = new sqlLoanEntity(loan);

        sqlUserEntity userRef = userRepo.getReferenceById(loan.getUserId());
        sqlBookEntity bookRef = bookRepo.getReferenceById(loan.getBookId());

        entity.setUser(userRef);
        entity.setBook(bookRef);

        return jpaRepo.save(entity).toLoan();
    }
}

```

(a) SQL Adapter

```

@Repository
@Profile("mongo")
public class mongoLoanAdapter implements LoanRepository {

    private final mongoLoanRepo loanRepo;

    public mongoLoanAdapter(mongoLoanRepo loanRepo) {
        this.loanRepo = loanRepo;
    }

    @Override
    public Loan save(Loan loan) {
        mongoLoanDoc document = new mongoLoanDoc(loan);
        return loanRepo.save(document).toLoan();
    }
}

```

(b) Mongo Adapter

Figure 1.5: Extending Generic Interface

1.4 Application Tier

Lastly, the Application Tier; Sandwiched in-between the two previous ones, this layer is the "brains" of the entire operation. This tier is responsible for the application's

actual functionality: processing the data, managing user inputs, performing security and bounds checking, everything.

Acting as a middle-man, or traffic coordinator, this layer facilitates the movement of data to and from the presentation and data tiers. If the presentation layer requests additional data, that request is received, handled, and processed by the application tier, with calls to the data layer being proxied by this layer too.

Crucially the data handled within this layer are generic, domain models with no context of where or how the processed data is consumed. The previously mentioned Repositories are used within this layer providing extremely simplified objects like the one seen in fig. 1.6 Data is then processed using these domain entities, and lastly translated into specific Data Transfer Objects (DTO), like the one in fig. 1.7. This is the process where data can start being combined, manipulated and shaped ready for consumption by the frontend; these become specially crafted bundles of information that the frontend then uses in its presentation. In a real application, these DTO "packets" can also be used to filter out or obfuscate internal only information; such as password hashes, backend flags, etc. While not used for this purpose in my implementation, the use of DTOs can be a factor in reducing sensitive information leakage.

```

public class Loan {
    private String id;
    private String userId;
    private String bookId;
    private LocalDate loanDate;
    private LocalDate dueDate;
    private LocalDate returnDate;
    private String status;

    private Double fineAmount;

    public String getId() {
        return id;
    }

    public String getUserId() {
        return userId;
    }

    public LocalDate getLoanDate() {
        return loanDate;
    }

    public LocalDate getDueDate() {
        return dueDate;
    }

    public String getBookId() {
        return bookId;
    }

    public LocalDate getReturnDate() {
        return returnDate;
    }

    public String getStatus() {
        return status;
    }

    public Double getFineAmount() {
        return fineAmount;
    }
}

```

Figure 1.6: Loan Domain Entity

```
public class LoanDto {  
    public String id;  
    public BookDto book;  
    public LocalDate loanDate;  
    public LocalDate dueDate;  
    public LocalDate returnDate;  
    public String status;  
    public Double fine;  
}
```

Figure 1.7: Loan DTO

1.4.1 Implementation

As the Application Tier is more of a blanket term for all business logic, this layer can be sub-divided into more specific sub-layers. In the implemented application this takes form through the "Controller" and "Service" packages. The "Config" and "Security" (appendices A.1 and A.5) packages could also be included under this tier; however their functions typically don't provide ongoing data manipulation.

Controllers

As mentioned previously, this implemented application is using the Spring framework for managing the actual web server communications; as a side effect many high level abstractions are present throughout the source code, with one such example being here in the controllers packages (fig. 1.8 and appendix A.2).

`@RequestMapping`, `@GetMapping`, and `@PostMapping` are all special annotations provided in this environment context to abstract away the setup and management of the API endpoints presented to outward services. The "Controller" classes as a whole are the traffic coordinators of the program, opening up communication "channels" that the presentation layers can use; however the controllers delegate processing these requests to the next sub-layer: Services.

Further note the appearance of both the domain `Loan` objects aswell as the functions returning `LoanDto` representations, within the below figure.

```

@RestController
@RequestMapping("/api/loans")
public class LoanController {
    private final LoanService loanService;
    private final UserRepository userRepo;

    public LoanController(
        LoanService loanService,
        UserRepository userRepository) {
        this.loanService = loanService;
        this.userRepo = userRepository;
    }

    private User getCurrentUser(Authentication auth) {
        String username = (String) auth.getPrincipal();
        return userRepo.findByUsername(username).orElseThrow();
    }

    @GetMapping("/me")
    public ResponseEntity<List<LoanDto>> myLoans(Authentication auth) {
        User user = getCurrentUser(auth);
        List<LoanDto> loans = loanService.findLoansByUser(user.getId());
        return ResponseEntity.ok(loans);
    }

    @PostMapping("/borrow")
    public ResponseEntity<?> borrow(
        @RequestBody Map<String, Object> body,
        Authentication auth) {
        User user = getCurrentUser(auth);
        String bookId = ((String) body.get(key: "bookId")).toString();
        try {
            Loan loan = loanService.borrowBook(user.getId(), bookId);
            return ResponseEntity.ok(loan);
        } catch (RuntimeException err) {
            return ResponseEntity.badRequest()
                .body(Map.of(key: "error", err.getMessage()));
        }
    }
}

```

Figure 1.8: LoanController Snippet

Services

Here the actual processing of requests and data manipulation occurs. fig. 1.9 shows the last remaining step within the Loan object cycle. Services are nothing more than a grouping of methods acting upon like objects.

The best demonstration of this pattern is the `toDto` method shown in the below figure. This method takes in a Loan domain object and converts it over to a DTO ready to be sent across to the presentation layer, and can even be seen in use near the bottom of the same figure.

Most of the actions taken by this method are just setting values of the new dto to the

previous `Loan`, however there are two additional blocks of logic. The first expands on the points made back in section 1.2; the logic that handles whether a loan is classed as "OVERDUE", and the subsequent fine is present here. This means no tampering can occur from frontend users, and all loan objects passed to presentation have relevant information "enriched" into the packets they receive. The frontend does not need to check if a loan is overdue, and then request the additional fine information as its all handled automatically.

The same is true with book details. It is not hard to imagine a system that is displaying loans to a user would also want access to the details about the books contained within said loans. As such `bookDto` objects are additionally embedded and checked before being transported to the presentation tier.

The controllers however, do not have to concern with any of that, the only methods called from controllers are the public `findLoansByUser` and `getLoanHistory`, which in turn handle all the translation and enrichment themselves. Controllers then finally receive whatever the outcome is of such service, and wraps it in a suitable `Response` to send back across to presentation.

```

@Service
public class LoanService {
    private final LoanRepository loanRepo;
    private final BookRepository bookRepo;
    private static final double FINE_ADDITION = 1.50;

    public LoanService(LoanRepository loanRepo,
                      BookRepository bookRepo,
                      UserRepository userRepo) {

        this.loanRepo = loanRepo;
        this.bookRepo = bookRepo;
    }

    private LoanDto toDto(Loan loan) {
        LoanDto dto = new LoanDto();

        dto.id = loan.getId();
        dto.loanDate = loan.getLoanDate();
        dto.returnDate = loan.getReturnDate();
        dto.dueDate = loan.getDueDate();

        if (loan.getStatus().equals(anObject: "BORROWED")
            && LocalDate.now().isAfter(loan.getDueDate())) {
            dto.status = "OVERDUE";

            long lateAmount = ChronoUnit.DAYS.between(
                loan.getDueDate(),
                LocalDate.now());
            dto.fine = lateAmount * FINE_ADDITION;
        } else {
            dto.status = loan.getStatus();
            dto.fine = loan.getFineAmount();
        }

        if (loan.getBookId() != null) {
            bookRepo.findById(loan.getBookId()).ifPresent(book -> {
                dto.book = BookDto.toDto(book);
            });
        }
        if (dto.book == null) {
            dto.book = new BookDto();
            dto.book.title = "ERROR";
            dto.book.author = "ERROR";
        }
        return dto;
    }

    public List<LoanDto> findLoansByUser(String userId) {
        List<Loan> loans = loanRepo.findByUserId(userId);
        return loans.stream().map(this::toDto).collect(Collectors.toList());
    }

    public List<LoanDto> getLoanHistory(String userId,
                                         LocalDate beginDate,
                                         LocalDate endDate) {
        List<Loan> loans = loanRepo.loanHistoryBetweenDates(userId, beginDate, endDate);
        return loans.stream().map(this::toDto).collect(Collectors.toList());
    }
}

```

Figure 1.9: LoanService Snippet

Chapter 2

Database Modelling Evaluation

In the modern technology landscape, there are two key schools of data storage methods. There are tried and true, rigid relational models; like SQLite, MariaDB, PostgreSQL which have powered software systems for decades. Then on the other hand, there are the more loose, and unstructured models that don't rely on relational transactions (No-SQL) such as MongoDB, Redis, and Cassandra. Picking the "right" model comes down to core design philosophies, scalability of the product and what types of data are being stored.

2.1 Relational (PostgreSQL)

Relational databases are the standard, typical implementation method of data storage, making use of a structured query language to define and manipulate data. This, as a concept, was first developed in the 1970s by Edgar Codd, and uses a series of rows and columns held within larger groupings called tables. (Foote 2023)

Each row present within a table holds an individual record, with a unique ID called the key, columns then hold the attributes of the data being stored. (Oracle 2021)

Relational DataBase Management Systems (RDBMS) are typically seen as persistent and slow for development purposes, due to their strict adherences to a schema-on-write architecture; before data can be stored, the "design" and layout of said data needs to be defined. Every single data type of every single column in every single table must be thoroughly considered before deployment. This methodical approach lends itself towards two of RDBMS core principles: **Normalization**, and **ACID**.

2.1.1 Normalization

Normalization is the process of organizing data in a database so that it is consistent, efficient, and easy to manage. (Nguyen 2023) Typically done in database systems to reduce redundancy (repeated data), and to breakup large expensive tables; there are many actions that can be taken to further normalize a databases structure.

As a demonstration of this; fig. 2.1 shows a simplified Entity-Relationship Diagram of the tables present in my applications implementation. Instead of storing all the data required in one large table, each general entity was separated out. This then meant objects like loans could reference a `userId` instead of storing the users name directly.

This reduces the chance of duplicate data errors and is a key benefit of the relational database models. Since Loan holds a references to a user object, if some detail within that table changes (e.g the username) only one value has to be updated. This update would then "propagate" throughout all uses of this field.

This however now introduces a trade off, reducing the complexity of each individual table increases the complexity of collecting data. To generate a human readable string like "`<username> borrowed <book> at <loanDate>`", now would require 3 separate database calls instead of a singular row; depending on how obsessively normalized the data becomes this could end up proving more computationally costly than having fewer simpler tables.

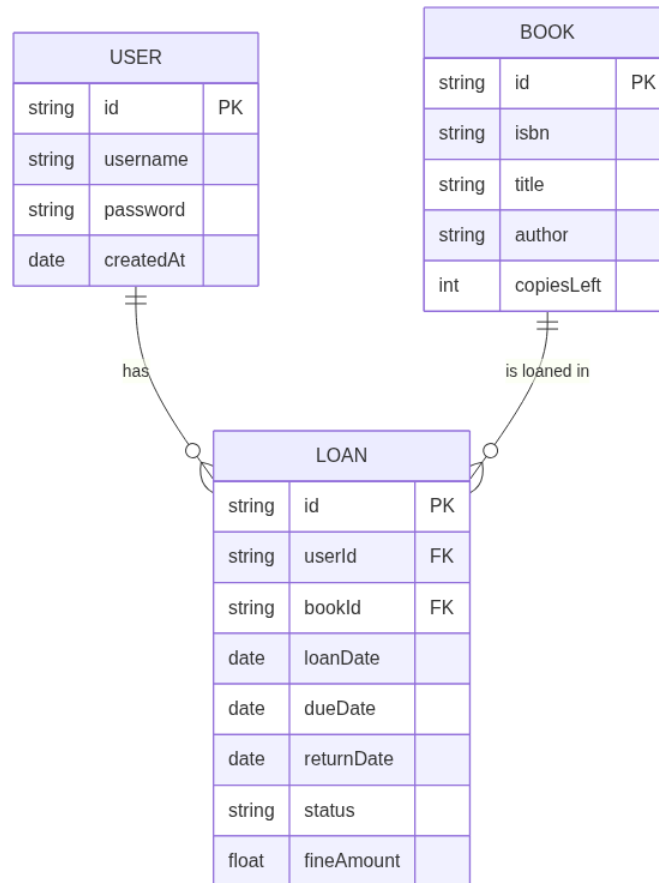


Figure 2.1: Simplified ERD

2.2 NoSQL (MongoDB)

In direct contrast to the rigid structure of relational databases, the NoSQL database methodology was developed in 1998, and "came about in direct response to the problems people were having with web data and a desire for faster processing". (Foote 2023) In every aspect, NoSQL databases like MongoDB are fundamentally inverses of traditional models; while RDBMS models focus on tables, with rows as entries and columns as fields, NoSQL databases instead utilize pre-existing flexible document types. MongoDB, for example, stores all its data in JSON-like objects called BSON. Each "Table" is now a self contained "Document" object, that opts to embed additional information instead of creating a static relationship to another

object.

NoSQL models also differ in their schemas. Defining and preplanning the entire database design from the beginning, was a hassle for ever changing environments like web applications; therefore NoSql completely disregards the structure and instead follows a Schema-on-read ideology. With the data stored, essentially as JSON objects, the structure of the data stores can change at will. These two databases models can be compared to development life cycle methods; relational models are identical to the processes taken with the waterfall model. Slow and methodical, with a great upfront time requirement. Whereas NoSQL models are more closely like agile methods, adopting a rapid iteration pattern letting developers fail fast and quickly refine what they need.

2.2.1 Usage in Implementation

While no physical evidence can be shown, this flexibility offered by MongoDB benefited the application greatly. During initial stages of development, before i had committed to a SQL design, mongoDB was used as the only operating database. This meant that i did not have to concern myself designing my application around the database and instead could focus on making the program itself, and the database structure kind of figured itself on its own. If i was missing a field, or wanted to add a new piece of data, i did not have to restart and up heave all my previous database work. fig. 2.2 shows the document structure used for the Loan objects. `fineAmount` is represented last as during development this field was absent, and was only added further down the line.

This structure created by just "using the application" was then copied over to the SQL schema. This flexibility let me design the database structure while actually using the database and finding pain points naturally.

```
@Document(collection = "loans")
public class mongoLoanDoc {
    @Id
    private String id;
    private String userId;
    private String bookId;

    private LocalDate loanDate;
    private LocalDate dueDate;
    private LocalDate returnDate;
    private String status;

    private Double fineAmount;

    public mongoLoanDoc() {
```

Figure 2.2: mongoLoanDoc Fields

Bibliography

- Foote, Keith (24th Aug. 2023). *NoSQL vs. SQL: Five Key Differences*. Dataversity. URL: <https://www.dataversity.net/articles/nosql-vs-sql-five-key-differences/> (visited on 21/12/2025).
- GeeksforGeeks (23rd July 2025). *Repository Design Pattern*. GeeksforGeeks. Section: Design Pattern. URL: <https://www.geeksforgeeks.org/system-design/repository-design-pattern/> (visited on 20/01/2026).
- Gillis, Alexander (22nd Oct. 2024). *What is a 3-Tier Application Architecture? — Definition from TechTarget*. TechTarget. URL: <https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application> (visited on 19/01/2026).
- Nguyen, Leah (18th July 2023). *A Brief Guide to Database Normalization*. Medium. URL: <https://medium.com/@ndleah/a-brief-guide-to-database-normalization-5ac59f093161> (visited on 06/01/2026).
- Oracle (18th June 2021). *What is a relational database?* Oracle. URL: <https://www.oracle.com/uk/database/what-is-a-relational-database/> (visited on 18/11/2025).

.1 Additional Figures

```
~/Coding/Java/DBA1-LibrarySystem/..
> .git
> backend
> .settings
> src
> main
> java
> github
> config
> controller
> dto
> model
> repository
> security
> service
> App.java
> resources
> test
> .classpath
> .project
> pom.xml
> frontend
> .vscode
> public
> src
> assets
> components
> lib
> app.css
> app.svelte
> main.js
> .gitignore
> index.html
> jsconfig.json
> package-lock.json
> package.json
> README.md
> svelte.config.js
> vite.config.js
```

Figure 3: Whole Application Structure

Appendix A

Backend Implementation

A.1 Configuration & Entry Point

Listing A.1: application.yaml

```
app:
spring:
  # profiles:
  #   active: sql
  application:
    name: backend
  datasource:
    url: jdbc:postgresql://localhost:5432/libdb
    username: libuser
    password: libpass
  jpa:
    hibernate:
      ddl-auto: update
      show-sql: false
  data:
    mongodb:
      uri: mongodb://localhost:27017/libdb
  flyway:
    enabled: false

server:
```

```

    port: 8082

jwt:
    secret: "123456789123456789123456789123456789123456789"
    expiration-ms: 86400000

cors:
    allowed-origins: http://localhost:5173

```

Listing A.2: redisConfig.java

```

package github.peaterpita.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.serializer.RedisSerializer;
import org.springframework.data.redis.serializer.StringRedisSerializer;

@Configuration
public class redisConfig {

    // #####
    // # Defining new redis configuration so that image data #
    // # (byte arrays) can be cached for 24 hours #
    // # This is used in the caching of cover images #
    // #####
    @Bean
    public RedisTemplate<String, byte[]> redisTemplate(
        RedisConnectionFactory connectFac) {
        RedisTemplate<String, byte[]> template = new RedisTemplate<>();
        template.setConnectionFactory(connectFac);

        template.setKeySerializer(new StringRedisSerializer());
        template.setValueSerializer(RedisSerializer.byteArray());

        return template;
    }
}

```

```
}  
}
```

Listing A.3: App.java

```
package github.peaterpita;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class App {  
    public static void main(String[] args) {  
        SpringApplication.run(App.class, args);  
    }  
}
```

A.2 Controllers

Listing A.4: AuthController.java

```
package github.peaterpita.controller;  
  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.PostMapping;  
import org.springframework.web.bind.annotation.RequestBody;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;  
  
import java.util.Map;  
import github.peaterpita.service.AuthService;  
  
@RestController  
@RequestMapping("/api/auth")  
public class AuthController {  
    private final AuthService authService;  
  
    public AuthController(AuthService authService) {  
        this.authService = authService;  
    }  
}
```

```

    }

    // #####
    // # Creating API endpoint '/api/auth/login'
    // # Login function takes two inputs, both present within
    // # the payload
    // # username and password
    // #
    // # Use authService attemptLogin function. If login
    // # successfull return success message with token
    // # else return 401 error
    // #####
    @PostMapping("/login")
    public ResponseEntity<?> login(
        @RequestBody Map<String, String> payload) {
        String username = payload.getDefault("username", "");
        String password = payload.getDefault("password", "");
        String token = authService.attemptLogin(username, password);

        ResponseEntity<?> res;
        if (token != null) {
            res = ResponseEntity.ok(Map.of(
                "success", true,
                "token", token));
        } else {
            res = ResponseEntity.status(401).body(Map.of(
                "success", false,
                "message", "Invalid username or password"));
        }
        return res;
    }
}

```

Listing A.5: BookController.java

```

package github.peaterpita.controller;

import java.net.URI;
import java.util.List;

```

```

import java.util.stream.Collectors;

import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import github.peaterpita.dto.BookDto;
import github.peaterpita.model.Book;
import github.peaterpita.service.BookService;
import github.peaterpita.service.CoverService;

@RestController
@RequestMapping("/api/books")
public class BookController {
    private final BookService bookService;
    private final CoverService coverService;

    public BookController(
        BookService bookService,
        CoverService coverService) {
        this.bookService = bookService;
        this.coverService = coverService;
    }

    // #####
    // # /api/books
    // # Get back all books that fulfill search query
    // # and available check
    // #####
    @GetMapping
    public ResponseEntity<List<BookDto>> searchBooks(
        @RequestParam(value = "query", required = false) String query,
        @RequestParam(value = "available", required = false) Boolean available) {

```

```

        List<Book> results = bookService.searchBooks(query, available);
        List<BookDto> dtos = results.stream().map(BookDto::toDto)
            .collect(Collectors.toList());
        return ResponseEntity.ok(dtos);
    }

    // #####
    // # /api/books/{bookId}
    // # use bookService to find a book with the passed in ID
    // # if book found return it
    // # else send error
    // #####
    @GetMapping("/{id}")
    public ResponseEntity<BookDto> getBook(@PathVariable String id) {

        try {
            Book book = bookService.findById(id);
            return ResponseEntity.ok(BookDto.toDto(book));
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

    // #####
    // # /api/books/cover/{isbn}
    // # Use custom getCover function on passed in isbn
    // # This function will either return a URI to a cached cover
    // # or a Open Library Retrieved cover
    // #####
    @GetMapping("/cover/{isbn}")
    public ResponseEntity<byte[]> getCover(@PathVariable String isbn) {

        byte[] image = coverService.getCover(isbn);

        if (image != null && image.length > 0) {
            return ResponseEntity.ok()
                .contentType(MediaType.IMAGE_JPEG).body(image);
        }
    }

```

```

        } else {
            return ResponseEntity.status(302)
                .location(URI.create("/src/assets/placeholder.png"))
                .build();
        }
    }
}

```

Listing A.6: LoanController.java

```

package github.peaterpita.controller;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

import org.apache.coyote.Response;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import github.peaterpita.dto.LoanDto;
import github.peaterpita.model.Loan;
import github.peaterpita.model.User;
import github.peaterpita.repository.UserRepository;
import github.peaterpita.service.LoanService;

@RestController
@RequestMapping("/api/loans")
public class LoanController {
    private final LoanService loanService;
    private final UserRepository userRepo;

```

```

public LoanController(
    LoanService loanService ,
    UserRepository userRepository) {
    this.loanService = loanService;
    this.userRepo = userRepository;
}

// #####
// # Helper function to get currently signed in user through
// # their JWT token.
// # JWT token can be reversed to get the users username.
// # This username can then be used
// # to get overall user object
// #####
private User getCurrentUser(Authentication auth) {
    String username = (String) auth.getPrincipal();
    return userRepo.findByUsername(username).orElseThrow();
}

// #####
// # Return back all loans associated with 'me' (currently
// # logged in user)
// #####
@GetMapping("/me")
public ResponseEntity<List<LoanDto>> myLoans(Authentication auth) {
    User user = getCurrentUser(auth);
    List<LoanDto> loans = loanService.findLoansByUser(user.getId());
    return ResponseEntity.ok(loans);
}

@PostMapping("/borrow")
public ResponseEntity<?> borrow(
    @RequestBody Map<String , Object> body,
    Authentication auth) {
    User user = getCurrentUser(auth);
    String bookId = ((String) body.get("bookId")).toString();
    try {

```

```

        Loan loan = loanService.borrowBook(user.getId(), bookId);
        return ResponseEntity.ok(loan);
    } catch (RuntimeException err) {
        return ResponseEntity.badRequest()
            .body(Map.of("error", err.getMessage()));
    }
}

@PostMapping("/return")
public ResponseEntity<?> returnBook(
    @RequestBody Map<String, String> body, Authentication auth) {
    User user = getCurrentUser(auth);

    String loanId = (body.get("loanId"));
    try {
        Loan loan = loanService.returnBook(user.getId(), loanId);
        return ResponseEntity.ok(loan);
    } catch (IllegalStateException err) {
        return ResponseEntity.badRequest()
            .body(Map.of("error", err.getMessage()));
    }
}

@PostMapping("/pay/{loanId}")
public ResponseEntity<?> payFine(
    @PathVariable String loanId,
    Authentication auth) {
    User user = getCurrentUser(auth);

    try {
        loanService.payFine(user.getId(), loanId);
        return ResponseEntity.ok(Map.of(
            "success", true,
            "message", "Paid"));
    } catch (RuntimeException err) {
        return ResponseEntity.badRequest()
            .body(Map.of("error", err.getMessage()));
    }
}

```

```

    }

    // #####
    // # /api/loans/history
    // # Two params passed in through headers. A beginning
    // # date and an end date
    // # This endpoint then returns all loans that occurred
    // # between those dates
    // #####
    @GetMapping("/history")
    public ResponseEntity<List<LoanDto>> getHistroy(
        @RequestParam LocalDate beginDate,
        @RequestParam LocalDate endDate,
        Authentication auth) {
        User user = getCurrentUser(auth);
        List<LoanDto> history = loanService.getLoanHistory(
            user.getId(),
            beginDate,
            endDate);
        return ResponseEntity.ok(history);
    }
}

```

A.3 Services

Listing A.7: AuthService.java

```

package github.peaterpita.service;

import java.util.Optional;

import org.springframework.stereotype.Service;

import github.peaterpita.model.User;
import github.peaterpita.repository.UserRepository;
import github.peaterpita.security.JwtUtil;

```

```

@Service
public class AuthService {
    private final UserRepository userRepo;
    private final JwtUtil jwtUtil;

    public AuthService(UserRepository userRepo, JwtUtil jwtUtil) {
        this.userRepo = userRepo;
        this.jwtUtil = jwtUtil;
    }

    // #####
    // # If user by passed in username exists and the passwords
    // # match
    // # generate a new JWT token and return to the user
    // #####
    public String attemptLogin(String username, String password) {
        System.out.println("——-LOGIN-ATTEMPT——");
        System.out.println("Input-Username: -[" + username + "]"");
        System.out.println("Input-Password: -[" + password + "]"");

        Optional<User> userOpt = userRepo.findByUsername(username);

        if (userOpt.isEmpty()) {
            return null;
        }

        User user = userOpt.get();

        if (user.getPassword() == null ||
            !user.getPassword().equals(password)) {
            return null;
        }

        return jwtUtil.generateToken(user.getUsername());
    }
}

```

Listing A.8: BookService.java

```
package github.peaterpita.service;

import java.util.List;
import java.util.stream.Collectors;

import org.springframework.stereotype.Service;

import github.peaterpita.model.Book;
import github.peaterpita.repository.BookRepository;

@Service
public class BookService {
    private final BookRepository bookRepo;

    public BookService(BookRepository bookRepo) {
        this.bookRepo = bookRepo;
    }

    public List<Book> findAll() {
        return bookRepo.findAll();
    }

    public Book findById(String id) {
        return bookRepo.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Error - finding - book" ));
    }

    public List<Book> searchByTitle(String title) {
        return bookRepo.findByTitle(title);
    }

    public List<Book> getAvailable() {
        return bookRepo.findByCopiesLeftGreaterThan(0);
    }

    public List<Book> searchBooks(String query, Boolean available) {
        List<Book> results = bookRepo.findAll();
```

```

    if (query != null && !query.isBlank()) {
        String lowerQuery = query.toLowerCase();
        results = results.stream()
            .filter(b -> b.getTitle()
                .toLowerCase()
                .contains(lowerQuery) ||
                b.getAuthor()
                    .toLowerCase()
                    .contains(lowerQuery))
            .collect(Collectors.toList());
    }

    if (Boolean.TRUE.equals(available)) {
        results = results.stream()
            .filter(b -> b.getCopiesLeft() != null
                && b.getCopiesLeft() > 0)
            .collect(Collectors.toList());
    }

    return results;
}
}

```

Listing A.9: LoanService.java

```

package github.peaterpita.service;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.stream.Collectors;

import org.springframework.stereotype.Service;

import github.peaterpita.dto.BookDto;
import github.peaterpita.dto.LoanDto;
import github.peaterpita.model.Book;

```

```

import github.peaterpita.model.Loan;
import github.peaterpita.repository.BookRepository;
import github.peaterpita.repository.LoanRepository;
import github.peaterpita.repository.UserRepository;

@Service
public class LoanService {
    private final LoanRepository loanRepo;
    private final BookRepository bookRepo;
    private static final double FINE_ADDITION = 1.50;

    public LoanService(LoanRepository loanRepo,
                       BookRepository bookRepo,
                       UserRepository userRepo) {

        this.loanRepo = loanRepo;
        this.bookRepo = bookRepo;
    }

    private LoanDto toDto(Loan loan) {
        LoanDto dto = new LoanDto();

        dto.id = loan.getId();
        dto.loanDate = loan.getLoanDate();
        dto.returnDate = loan.getReturnDate();
        dto.dueDate = loan.getDueDate();

        if (loan.getStatus().equals("BORROWED")
            && LocalDate.now().isAfter(loan.getDueDate())) {
            dto.status = "OVERDUE";

            long lateAmount = ChronoUnit.DAYS.between(
                loan.getDueDate(),
                LocalDate.now());
            dto.fine = lateAmount * FINE_ADDITION;
        } else {
            dto.status = loan.getStatus();
            dto.fine = loan.getFineAmount();
        }
    }
}

```

```

    }

    if (loan.getBookId() != null) {
        bookRepo.findById(loan.getBookId()).ifPresent(book -> {
            dto.book = BookDto.toDto(book);
        });
    }
    if (dto.book == null) {
        dto.book = new BookDto();
        dto.book.title = "ERROR";
        dto.book.author = "ERROR";
    }
    return dto;
}

public List<LoanDto> findLoansByUser(String userId) {
    List<Loan> loans = loanRepo.findById(userId);
    return loans.stream().map(this::toDto).collect(Collectors.toList());
}

public List<LoanDto> getLoanHistory(String userId,
    LocalDate beginDate,
    LocalDate endDate) {
    List<Loan> loans = loanRepo.loanHistoryBetweenDates(userId, beginDate, endDate);
    return loans.stream().map(this::toDto).collect(Collectors.toList());
}

public Loan borrowBook(String userId, String bookId) {
    Book book = bookRepo.findById(bookId)
        .orElseThrow(() -> new RuntimeException("No-Book"));

    if (book.getCopiesLeft() <= 0) {
        throw new RuntimeException("none-left");
    }

    if (loanRepo.doesUserHaveActiveLoanForBookId(userId, bookId)) {
        throw new RuntimeException("You-already-have-this-book-out-on-loan");
    }
}

```

```

        Loan loan = new Loan();
        loan.setBookId(bookId);
        loan.setUserId(userId);
        loan.setLoanDate(LocalDate.now());
        loan.setDueDate(LocalDate.now().plusDays(14));
        loan.setStatus("BORROWED"); // TODO: USE ENUM
        loan.setFineAmount(0.0);

        book.setCopiesLeft(book.getCopiesLeft() - 1);
        bookRepo.save(book);

        return loanRepo.save(loan);
    }

    public Loan returnBook(String userId, String loanId) {
        Loan loan = loanRepo.findById(loanId)
            .orElseThrow(() -> new RuntimeException("No-Loan"));

        if (!(loan.getStatus().equals("BORROWED"))) {
            throw new RuntimeException("Already-paid-or-returned-or-lost");
        }

        // #####
        // # Ensure DueDate constraints on trusted backend server.
        // # Prevents untrusted users from bypassing payment
        // #requirements by returning book as if it was a normal book.
        // #####
        if (LocalDate.now().isAfter(loan.getDueDate())) {
            throw new RuntimeException("Book-cannot-be-returned");
        }

        loan.setReturnDate(LocalDate.now());
        loan.setStatus("RETURNED");

        Book book = bookRepo.findById(loan.getBookId())
            .orElseThrow(() -> new RuntimeException("Bad-error"));
        book.setCopiesLeft(

```

```

        (book.getCopiesLeft() == null ? 0 : book.getCopiesLeft()) + 1);

    bookRepo.save(book);
    return loanRepo.save(loan);

}

public Loan payFine(String userId, String loanId) {
    Loan loan = loanRepo.findById(loanId)
        .orElseThrow(() -> new RuntimeException("Loan was not found"));

    if (!loan.getUserId()
        .equals(userId))
        throw new RuntimeException(
            "Cannot pay off another users loan");

    // #####
    // # Although present on the loan, recalculate the fine
    // # amount needed. Between user clicking pay and
    // # them actually getting here, time could have ticked
    // # over to a new day. Should really compare
    // # new price vs the one stored on the loan.
    // #####
    long lateAmount = ChronoUnit.DAYS.between(loan.getDueDate(), LocalDate.now());
    double fine = lateAmount * FINE_ADDITION;

    loan.setFineAmount(fine);
    loan.setReturnDate(LocalDate.now());
    loan.setStatus("PAID");

    // TODO: Change to method DRY
    Book book = bookRepo.findById(loan.getBookId())
        .orElseThrow(() -> new RuntimeException("Bad error"));
    book.setCopiesLeft(
        (book.getCopiesLeft() == null ? 0 : book.getCopiesLeft()) + 1);

    bookRepo.save(book);
    return loanRepo.save(loan);
}

```

```

    }
}

```

Listing A.10: CoverService.java

```

package github.peaterpita.service;

import java.time.Duration;

import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestTemplate;

@Service
public class CoverService {
    private final RedisTemplate<String, byte[]> imgTemplate;
    private final RestTemplate restTemplate;

    public CoverService(RedisTemplate<String, byte[]> imgTemplate) {
        this.imgTemplate = imgTemplate;
        this.restTemplate = new RestTemplate();
    }

    public byte[] getCover(String isbn) {

        if (isbn == null || isbn.isBlank()) {
            return null;
        }

        String cacheKey = "cover:" + isbn;

        // #####
        // # Check if cover is cached in Redis first. If it is use
        // # that
        // #####

```

```

if (Boolean.TRUE.equals(imgTemplate.containsKey(cacheKey))) {
    return imgTemplate.opsForValue().get(cacheKey);
}

try {

    // #####
    // # If the cover is not cached, reach out to the
    // # openlibrary covers api to get a cover for that isbn
    // # If a reply is recieved and the headers are good
    // # status, extract body
    // # and cache contents into redis.
    // # return same contents out
    // #####
    String url = "https://covers.openlibrary.org/b/isbn/"
        + isbn
        + "-M.jpg?default=false";
    ResponseEntity<byte[]> res = restTemplate.getForEntity(url, byte[].class);

    if (res.getStatusCode().is2xxSuccessful() && res.getBody() != null) {
        byte[] imgBytes = res.getBody();
        imgTemplate.opsForValue().set(cacheKey, imgBytes, Duration.ofHours(24));
        return imgBytes;
    }
} catch (HttpClientErrorException.NotFound err) {

    // #####
    // # Even if the request to OpenLibrary fails,
    // # still cache a result. This prevent covers that
    // # continue failing to keep calling the same endpoint.
    // # These caches only last 24 hours as book covers may be
    // # updated / be temporarily down
    // #####

    imgTemplate.opsForValue().set(cacheKey, new byte[0], Duration.ofHours(24));
} catch (Exception err) {
    System.out.println("General Error in the getCover");
}

```

```
        return null;
    }

}
```

A.4 Domain Models & DTOs

Listing A.11: Book.java

```
package github.peaterpita.model;

public class Book {
    private String id;
    private String isbn;
    private String title;
    private String author;
    private Integer copiesLeft = 1;

    public String getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public String getIsbn() {
        return isbn;
    }

    public String getAuthor() {
        return author;
    }

    public Integer getCopiesLeft() {
        return copiesLeft;
    }

    // #####
}
```

```

// # Setters #
// #####

public void setId(String id) {
    this.id = id;
}

public void setTitle(String title) {
    this.title = title;
}

public void setIsbn(String isbn) {
    this.isbn = isbn;
}

public void setAuthor(String author) {
    this.author = author;
}

public void setCopiesLeft(Integer copiesLeft) {
    this.copiesLeft = copiesLeft;
}
}

```

Listing A.12: BookDto.java

```

package github.peaterpita.dto;

import github.peaterpita.model.Book;

public class BookDto {
    public String id;
    public String isbn;
    public String title;
    public String author;
    public String coverURL;
    public Integer copiesLeft;
}

```

```

public static BookDto toDto(Book book) {
    BookDto dto = new BookDto();

    dto.id = book.getId();
    dto.isbn = book.getIsbn();
    dto.title = book.getTitle();
    dto.author = book.getAuthor();
    dto.copiesLeft = book.getCopiesLeft();

    // #####
    // # If no ISBN present (never should occur)
    // # set coverURL to a placeholder image stored on the
    // # frontend
    // #####
    if (book.getIsbn() != null) {
        dto.coverURL = "/api/books/cover/" + book.getIsbn();
    } else {
        dto.coverURL = "/assets/placeholder.png";
    }

    return dto;
}
}

```

Listing A.13: Loan.java

```

package github.peaterpita.model;

import java.time.LocalDate;

public class Loan {
    private String id;
    private String userId;
    private String bookId;
    private LocalDate loanDate;
    private LocalDate dueDate;
}

```

```
private LocalDate returnDate;
private String status;

private Double fineAmount;

public String getId() {
    return id;
}

public String getUserId() {
    return userId;
}

public LocalDate getLoanDate() {
    return loanDate;
}

public LocalDate getDueDate() {
    return dueDate;
}

public String getBookId() {
    return bookId;
}

public LocalDate getReturnDate() {
    return returnDate;
}

public String getStatus() {
    return status;
}

public Double getFineAmount() {
    return fineAmount;
}

// #####
```

```
// # Setters #  
// #####
```

```
public void setId(String Id) {  
    this.id = Id;  
}
```

```
public void setBookId(String bookId) {  
  
    this.bookId = bookId;  
}
```

```
public void setUserId(String userId) {  
    this.userId = userId;  
}
```

```
public void setLoanDate(LocalDate loanDate) {  
    this.loanDate = loanDate;  
}
```

```
public void setDueDate(LocalDate dueDate) {  
    this.dueDate = dueDate;  
}
```

```
public void setReturnDate(LocalDate returnDate) {  
    this.returnDate = returnDate;  
}
```

```
public void setStatus(String status) {  
    this.status = status;  
}
```

```
public void setFineAmount(Double amount) {  
    this.fineAmount = amount;  
}
```

```
}
```

```
package github.peaterpita.dto;

import java.time.LocalDate;

public class LoanDto {
    public String id;
    public BookDto book;
    public LocalDate loanDate;
    public LocalDate dueDate;
    public LocalDate returnDate;
    public String status;
    public Double fine;
}
```

Listing A.15: User.java

```
package github.peaterpita.model;

import java.time.LocalDateTime;

public class User {
    private String id;
    private String username;
    private String password;
    private LocalDateTime createdAt;

    public String getId() {
        return id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
}
```

```

    public String getPassword() {
        return password;
    }

    // #####
    // # Setters #
    // #####

    public void setPassword(String password) {
        this.password = password;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime date) {
        this.createdAt = date;
    }

    public void setId(String id) {
        this.id = id;
    }
}

```

A.5 Security Configuration

Listing A.16: SecurityConfig.java

```

package github.peaterpita.security;

import java.util.List;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

```

```

import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;

```

@Configuration

```

public class SecurityConfig {

```

```

    private final JwtUtil jwtUtil;
    private final List<String> allowedOrigins;

```

```

    public SecurityConfig(JwtUtil jwtUtil,
        @Value("${cors.allowed-origins}") List<String> allowedOrigins) {
        this.jwtUtil = jwtUtil;
        this.allowedOrigins = allowedOrigins;
    }

```

@Bean

```

    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        var jwtFilter = new JwtAuthenticationFilter(jwtUtil);

        http.csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(request -> {
                var cfg = new CorsConfiguration();
                cfg.setAllowedOrigins(allowedOrigins);
                cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
                cfg.setAllowedHeaders(List.of("*"));
                cfg.setAllowCredentials(true);
                return cfg;
            })))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/books/cover/**").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        // .httpBasic(Customizer.withDefaults());
    }

```

```

        return http.build();
    }
}

```

Listing A.17: JwtUtil.java

```

package github.peaterpita.security;

import java.security.Key;
import java.util.Date;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;

@Component
public class JwtUtil {

    private final Key key;
    private final long expirationMs;

    public JwtUtil(
        @Value("${jwt.secret}") String secret,
        @Value("${jwt.expiration-ms}") long expirationMsEnv) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes());
        this.expirationMs = expirationMsEnv;
    }

    public String generateToken(String username) {
        Date now = new Date();
        Date expir = new Date(now.getTime() + expirationMs);
    }
}

```

```

        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(now)
            .setExpiration(expir)
            .signWith(key, io.jsonwebtoken.SignatureAlgorithm.HS256)
            .compact();
    }

    public String getUsernameFromToken(String token) throws JwtException {
        return Jwts.parserBuilder()
            .setSigningKey(key)
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token);
            return true;
        } catch (JwtException e) {
            return false;
        }
    }
}

```

Listing A.18: JwtAuthenticationFilter.java

```

package github.peaterpita.security;

import java.io.IOException;

import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.util.StringUtils;

```

```

import org.springframework.web.filter.OncePerRequestFilter;

import java.util.List;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtUtil jwtUtil;

    public JwtAuthenticationFilter(JwtUtil jwtUtil) {
        this.jwtUtil = jwtUtil;
    }

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

        String header = request.getHeader("Authorization");
        String token = null;

        if (StringUtils.hasText(header) && header.startsWith("Bearer ")) {
            token = header.substring(7);
        }

        if (token != null && jwtUtil.validateToken(token)) {
            String username = jwtUtil.getUsernameFromToken(token);
            var auth = new UsernamePasswordAuthenticationToken(username, null,
                List.of(new SimpleGrantedAuthority("ROLE_USER")));
            SecurityContextHolder.getContext().setAuthentication(auth);
        }

        filterChain.doFilter(request, response);
    }
}

```

}

}

Appendix B

Data Persistence Layer

B.1 Repository Interfaces

Listing B.1: BookRepository.java

```
package github.peaterpita.repository;

import java.util.List;
import java.util.Optional;

import github.peaterpita.model.Book;

public interface BookRepository {

    Book save(Book book);

    List<Book> findAll();

    Optional<Book> findById(String id);

    List<Book> findByTitle(String title);

    List<Book> findByAuthor(String author);

    List<Book> findByCopiesLeftGreaterThan(int copies);
```

```
}
```

Listing B.2: LoanRepository.java

```
package github.peaterpita.repository;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

import github.peaterpita.model.Loan;

public interface LoanRepository {
    Loan save(Loan loan);

    Optional<Loan> findById(String loanId);

    List<Loan> findById(String userId);

    List<Loan> loanHistoryBetweenDates(String userId, LocalDate beginDate, LocalDate endDate);

    boolean doesUserHaveActiveLoanForBookId(String userId, String bookId);
}
```

Listing B.3: UserRepository.java

```
package github.peaterpita.repository;

import github.peaterpita.model.User;
import java.util.Optional;

public interface UserRepository {
    User save(User user);

    Optional<User> findByUsername(String username);
}
```

B.2 SQL Implementation

B.2.1 Adapters

Listing B.4: sqlBookAdapter.java

```
package github.peaterpita.repository.sql;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Repository;

import github.peaterpita.model.Book;
import github.peaterpita.repository.BookRepository;

@Repository
@Profile("sql")
public class sqlBookAdapter implements BookRepository {

    private final JpaBookRepo jpaRepo;

    public sqlBookAdapter(JpaBookRepo jpaRepo) {
        this.jpaRepo = jpaRepo;
    }

    @Override
    public Book save(Book book) {
        sqlBookEntity entity = new sqlBookEntity(book);
        return jpaRepo.save(entity).toBook();
    }

    @Override
    public List<Book> findAll() {
        return jpaRepo.findAll().stream().map(sqlBookEntity::toBook)
            .collect(Collectors.toList());
    }
}
```

```

@Override
public Optional<Book> findById(String id) {
    return jpaRepo.findById(id).map(sqlBookEntity::toBook);
}

@Override
public List<Book> findByTitle(String title) {
    return jpaRepo.findByTitleContainingIgnoreCase(title)
        .stream().map(sqlBookEntity::toBook)
        .collect(Collectors.toList());
}

@Override
public List<Book> findByAuthor(String author) {
    return jpaRepo.findByAuthorContainingIgnoreCase(author)
        .stream().map(sqlBookEntity::toBook)
        .collect(Collectors.toList());
}

@Override
public List<Book> findByCopiesLeftGreaterThan(int copies) {
    return jpaRepo.findByCopiesLeftGreaterThan(copies)
        .stream().map(sqlBookEntity::toBook)
        .collect(Collectors.toList());
}
}

```

Listing B.5: sqlLoanAdapter.java

```

package github.peaterpita.repository.sql;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

import org.springframework.context.annotation.Profile;

```

```

import org.springframework.stereotype.Repository;

import github.peaterpita.model.Loan;
import github.peaterpita.repository.LoanRepository;

@Repository
@Profile("sql")
public class sqlLoanAdapter implements LoanRepository {

    private final jpaLoanRepo jpaRepo;

    private final jpaUserRepo userRepo;
    private final jpaBookRepo bookRepo;

    public sqlLoanAdapter(jpaLoanRepo loanRepo,
        jpaUserRepo userRepo,
        jpaBookRepo bookRepo) {
        this.jpaRepo = loanRepo;
        this.userRepo = userRepo;
        this.bookRepo = bookRepo;
    }

    @Override
    public Loan save(Loan loan) {
        sqlLoanEntity entity = new sqlLoanEntity(loan);

        sqlUserEntity userRef = userRepo.getReferenceById(loan.getUserId());
        sqlBookEntity bookRef = bookRepo.getReferenceById(loan.getBookId());

        entity.setUser(userRef);
        entity.setBook(bookRef);

        return jpaRepo.save(entity).toLoan();
    }

    @Override
    public List<Loan> findByUserId(String userId) {
        return jpaRepo.findByUserId(userId).stream()

```

```

        .map(sqlLoanEntity::toLoan)
        .collect(Collectors.toList());
    }

    @Override
    public List<Loan> loanHistoryBetweenDates(String userId ,
        LocalDate beginDate ,
        LocalDate endDate) {
        return jpaRepo.findHistoryBetween(userId , beginDate , endDate)
            .stream()
            .map(sqlLoanEntity::toLoan)
            .collect(Collectors.toList());
    }

    @Override
    public Optional<Loan> findById(String loanId) {
        return jpaRepo.findById(loanId).map(sqlLoanEntity::toLoan);
    }

    @Override
    public boolean doesUserHaveActiveLoanForBookId(String userId , String bookId) {
        return jpaRepo.existsByUserIdAndBookIdAndReturnDateIsNull(userId , bookId);
    }
}

```

Listing B.6: sqlUserAdapter.java

```

package github.peaterpita.repository.sql;

import java.util.Optional;

import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Repository;

import github.peaterpita.model.User;
import github.peaterpita.repository.UserRepository;

@Repository

```

```

@Profile("sql")
public class sqlUserAdapter implements UserRepository {
    private final jpaUserRepo userRepo;

    public sqlUserAdapter(jpaUserRepo userRepo) {
        this.userRepo = userRepo;
    }

    @Override
    public User save(User user) {
        sqlUserEntity entity = new sqlUserEntity(user);
        return userRepo.save(entity).toUser();
    }

    @Override
    public Optional<User> findByUsername(String username) {
        return userRepo.findByUsername(username).map(sqlUserEntity::toUser);
    }
}

```

B.2.2 Entities

Listing B.7: sqlBookEntity.java

```

package github.peaterpita.repository.sql;

import java.util.UUID;

import javax.persistence.Column;

import github.peaterpita.model.Book;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "books")
public class sqlBookEntity {

```

```

@Id
private String id;

@Column(unique = true)
private String isbn;

@Column
private String title;

@Column
private String author;

@Column(name = "copies_left")
private Integer copiesLeft;

public sqlBookEntity() {
}

public sqlBookEntity(Book book) {
    if (book.getId() != null) {
        this.id = book.getId();
    } else {
        this.id = UUID.randomUUID().toString();
    }
    this.isbn = book.getIsbn();
    this.title = book.getTitle();
    this.author = book.getAuthor();
    this.copiesLeft = book.getCopiesLeft();
}

public Book toBook() {
    Book book = new Book();

    book.setId(this.id);
    book.setIsbn(this.isbn);
    book.setTitle(this.title);
    book.setAuthor(this.author);
    book.setCopiesLeft(this.copiesLeft);
}

```

```

        return book;
    }

    public String getId() {
        return id;
    }
}

```

Listing B.8: sqlLoanEntity.java

```

package github.peaterpita.repository.sql;

import java.time.LocalDate;
import java.util.UUID;

import javax.persistence.JoinColumn;

import github.peaterpita.model.Loan;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

@Entity
@Table(name = "loans")
public class sqlLoanEntity {

    @Id
    private String id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private sqlUserEntity user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "book_id", nullable = false)

```

```

private sqlBookEntity book;

private LocalDate loanDate;
private LocalDate dueDate;
private LocalDate returnDate;

private String status;
private Double fineAmount;

public sqlLoanEntity() {
}

public sqlLoanEntity(Loan loan) {
    if (loan.getId() != null) {
        this.id = loan.getId();
    } else {
        this.id = UUID.randomUUID().toString();
    }

    this.loanDate = loan.getLoanDate();
    this.dueDate = loan.getDueDate();
    this.returnDate = loan.getReturnDate();

    this.status = loan.getStatus();
    this.fineAmount = loan.getFineAmount();
}

public Loan toLoan() {
    Loan loan = new Loan();
    loan.setId(this.id);

    if (this.user != null)
        loan.setUserId(this.user.getId());

    if (this.book != null)
        loan.setBookId(this.book.getId());
}

```

```

        loan.setDueDate(this.dueDate);
        loan.setLoanDate(this.loanDate);
        loan.setReturnDate(this.returnDate);

        loan.setStatus(this.status);
        loan.setFineAmount(this.fineAmount);

        return loan;
    }

    public void setUser(sqlUserEntity user) {
        this.user = user;
    }

    public void setBook(sqlBookEntity book) {
        this.book = book;
    }
}

```

Listing B.9: sqlUserEntity.java

```

package github.peaterpita.repository.sql;

import java.time.LocalDateTime;
import java.util.UUID;

import javax.persistence.Column;

import github.peaterpita.model.User;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "users")
public class sqlUserEntity {

    @Id
    private String id;

```

```
@Column(unique = true, nullable = false)
private String username;

private String password;
private LocalDateTime createdAt;

public sqlUserEntity() {
};

public sqlUserEntity(User user) {
    if (user.getId() != null) {
        this.id = user.getId();
    } else {
        this.id = UUID.randomUUID().toString();
    }

    this.username = user.getUsername();
    this.password = user.getPassword();
    this.createdAt = user.getCreatedAt();
}

public User toUser() {
    User user = new User();

    user.setId(this.id);
    user.setUsername(this.username);
    user.setPassword(this.password);

    user.setCreatedAt(this.createdAt);

    return user;
}

public String getId() {
    return id;
}
```

```
}
```

B.2.3 Internal Repositories

Listing B.10: jpaBookRepo.java

```
package github.peaterpita.repository.sql;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

public interface jpaBookRepo extends JpaRepository<sqlBookEntity, String> {

    List<sqlBookEntity> findByTitleContainingIgnoreCase(String title);

    List<sqlBookEntity> findByAuthorContainingIgnoreCase(String author);

    List<sqlBookEntity> findByCopiesLeftGreaterThan(int copies);

    List<sqlBookEntity> findByTitle(String title);
}
```

Listing B.11: jpaLoanRepo.java

```
package github.peaterpita.repository.sql;

import java.time.LocalDate;
import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface jpaLoanRepo extends JpaRepository<sqlLoanEntity, String> {

    List<sqlLoanEntity> findById(String userId);

    @Query("SELECT loan FROM sqlLoanEntity loan WHERE loan.user.id = :userId")
```

```

        + "AND loan.loanDate BETWEEN :beginDate AND :endDate")
List<sqlLoanEntity> findHistoryBetween(
    @Param("userId") String userId,
    @Param("beginDate") LocalDate beginDate,
    @Param("endDate") LocalDate endDate);

    boolean existsByUserIdAndBookIdAndReturnDateIsNull(String userId, String bookId);
}

```

Listing B.12: jpaUserRepo.java

```

package github.peaterpita.repository.sql;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

public interface jpaUserRepo extends JpaRepository<sqlUserEntity, String> {
    Optional<sqlUserEntity> findByUsername(String username);
}

```

B.3 MongoDB Implementation

B.3.1 Adapters

Listing B.13: mongoBookAdapter.java

```

package github.peaterpita.repository.mongo;

import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Repository;

import github.peaterpita.model.Book;
import github.peaterpita.repository.BookRepository;

@Repository

```

```
@Profile("mongo")
public class mongoBookAdapter implements BookRepository {
    private final mongoBookRepo mongoRepo;

    public mongoBookAdapter(mongoBookRepo mongoRepo) {
        this.mongoRepo = mongoRepo;
    }

    @Override
    public Book save(Book book) {
        mongoBookDoc document = new mongoBookDoc(book);
        return mongoRepo.save(document).toBook();
    }

    @Override
    public List<Book> findAll() {
        return mongoRepo.findAll().stream()
            .map(mongoBookDoc::toBook).collect(Collectors.toList());
    }

    @Override
    public Optional<Book> findById(String id) {
        return mongoRepo.findById(id).map(mongoBookDoc::toBook);
    }

    @Override
    public List<Book> findByTitle(String title) {
        return mongoRepo.findByTitleContainingIgnoreCase(title).stream()
            .map(mongoBookDoc::toBook)
            .collect(Collectors.toList());
    }

    @Override
    public List<Book> findByAuthor(String author) {
        return mongoRepo.findByAuthorContainingIgnoreCase(author).stream()
            .map(mongoBookDoc::toBook)
            .collect(Collectors.toList());
    }
}
```

```

    @Override
    public List<Book> findByCopiesLeftGreaterThan(int copies) {
        return mongoRepo.findByCopiesLeftGreaterThan(copies).stream()
            .map(mongoBookDoc::toBook)
            .collect(Collectors.toList());
    }
}

```

Listing B.14: mongoLoanAdapter.java

```

package github.peaterpita.repository.mongo;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Repository;

import github.peaterpita.model.Loan;
import github.peaterpita.repository.LoanRepository;

@Repository
@Profile("mongo")
public class mongoLoanAdapter implements LoanRepository {

    private final mongoLoanRepo loanRepo;

    public mongoLoanAdapter(mongoLoanRepo loanRepo) {
        this.loanRepo = loanRepo;
    }

    @Override
    public Loan save(Loan loan) {
        mongoLoanDoc document = new mongoLoanDoc(loan);
        return loanRepo.save(document).toLoan();
    }
}

```

```

    }

    @Override
    public List<Loan> findByUserId(String userId) {
        return loanRepo.findByUserId(userId).stream()
            .map(mongoLoanDoc::toLoan).collect(Collectors.toList());
    }

    @Override
    public List<Loan> loanHistoryBetweenDates(
        String userId,
        LocalDate beginDate,
        LocalDate endDate) {
        return loanRepo.findHistory(userId, beginDate, endDate).stream()
            .map(mongoLoanDoc::toLoan)
            .collect(Collectors.toList());
    }

    @Override
    public Optional<Loan> findById(String loanId) {
        return loanRepo.findById(loanId).map(mongoLoanDoc::toLoan);
    }

    @Override
    public boolean doesUserHaveActiveLoanForBookId(String userId, String bookId) {
        return loanRepo.existsByUserIdAndBookIdAndReturnDateIsNull(userId, bookId);
    }
}

```

Listing B.15: mongoUserAdapter.java

```

package github.peaterpita.repository.mongo;

import java.util.Optional;

import org.springframework.context.annotation.Profile;
import org.springframework.stereotype.Repository;

import github.peaterpita.model.User;

```

```

import github.peaterpita.repository.UserRepository;

@Repository
@Profile("mongo")
public class mongoUserAdapter implements UserRepository {

    private final mongoUserRepo userRepo;

    public mongoUserAdapter(mongoUserRepo userRepo) {
        this.userRepo = userRepo;
    }

    @Override
    public User save(User user) {
        mongoUserDoc document = new mongoUserDoc(user);
        return userRepo.save(document).toUser();
    }

    @Override
    public Optional<User> findByUsername(String username) {
        return userRepo.findByUsername(username).map(mongoUserDoc::toUser);
    }
}

```

B.3.2 Documents

Listing B.16: mongoBookDoc.java

```

package github.peaterpita.repository.mongo;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;

import github.peaterpita.model.Book;

@Document(collection = "books")
public class mongoBookDoc {
    @Id

```

```

private String id;

private String isbn;
private String title;
private String author;

@Field("copies_left")
private Integer copiesLeft;

public mongoBookDoc() {
}

public mongoBookDoc(Book book) {
    this.id = book.getId();
    this.isbn = book.getIsbn();
    this.title = book.getTitle();
    this.author = book.getAuthor();
    this.copiesLeft = book.getCopiesLeft();
}

public Book toBook() {
    Book book = new Book();

    book.setId(this.id);
    book.setIsbn(this.isbn);
    book.setTitle(this.title);
    book.setAuthor(this.author);
    book.setCopiesLeft(this.copiesLeft);

    return book;
}
}

```

Listing B.17: mongoLoanDoc.java

```

package github.peaterpita.repository.mongo;

import java.time.LocalDate;

```

```

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import github.peaterpita.model.Loan;

@Document(collection = "loans")
public class mongoLoanDoc {
    @Id
    private String id;
    private String userId;
    private String bookId;

    private LocalDate loanDate;
    private LocalDate dueDate;
    private LocalDate returnDate;
    private String status;

    private Double fineAmount;

    public mongoLoanDoc() {

    }

    public mongoLoanDoc(Loan loan) {
        this.id = loan.getId();
        this.userId = loan.getUserId();
        this.bookId = loan.getBookId();

        this.loanDate = loan.getLoanDate();
        this.dueDate = loan.getDueDate();
        this.returnDate = loan.getReturnDate();
        this.status = loan.getStatus();
        this.fineAmount = loan.getFineAmount();
    }

    public Loan toLoan() {
        Loan loan = new Loan();

```

```

        loan.setId(this.id);
        loan.setBookId(this.bookId);
        loan.setUserId(this.userId);
        loan.setDueDate(this.dueDate);
        loan.setLoanDate(this.loanDate);
        loan.setReturnDate(this.returnDate);

        loan.setStatus(this.status);
        loan.setFineAmount(this.fineAmount);

        return loan;
    }
}

```

Listing B.18: mongoUserDoc.java

```

package github.peaterpita.repository.mongo;

import java.time.LocalDateTime;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import github.peaterpita.model.User;

@Document(collection = "users")
public class mongoUserDoc {
    @Id
    private String id;
    private String username;
    private String password;
    private LocalDateTime createdAt;

    public mongoUserDoc() {
    }

    public mongoUserDoc(User user) {
        this.id = user.getId();
    }
}

```

```

        this.username = user.getUsername();
        this.password = user.getPassword();
        this.createdAt = user.getCreatedAt();
    }

    public User toUser() {
        User user = new User();

        user.setId(this.id);
        user.setUsername(this.username);
        user.setPassword(this.password);
        user.setCreatedAt(this.createdAt);

        return user;
    }
}

```

B.3.3 Internal Repositories

Listing B.19: mongoBookRepo.java

```

package github.peaterpita.repository.mongo;

import java.util.List;

import org.springframework.data.mongodb.repository.MongoRepository;

public interface mongoBookRepo extends MongoRepository<mongoBookDoc, String> {
    List<mongoBookDoc> findByTitle(String title);

    List<mongoBookDoc> findByTitleContainingIgnoreCase(String title);

    List<mongoBookDoc> findByAuthorContainingIgnoreCase(String author);

    List<mongoBookDoc> findByCopiesLeftGreaterThan(int copies);
}

```

Listing B.20: mongoLoanRepo.java

```

package github.peaterpita.repository.mongo;

```

```

import java.time.LocalDate;
import java.util.List;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.data.mongodb.repository.Query;

public interface mongoLoanRepo extends MongoRepository<mongoLoanDoc, String> {

    List<mongoLoanDoc> findByUserId(String userId);

    @Query("{- 'userId' : -?0, - 'loanDate' : - { $gte : -?1, - $lt : -?2 } -}")
    List<mongoLoanDoc> findHistory(String userId, LocalDate beginDate, LocalDate endDate);

    boolean existsByUserIdAndBookIdAndReturnDateIsNull(String userId, String bookId);
}

```

Listing B.21: mongoUserRepo.java

```

package github.peaterpita.repository.mongo;

import java.util.Optional;

import org.springframework.data.mongodb.repository.MongoRepository;

public interface mongoUserRepo extends MongoRepository<mongoUserDoc, String> {
    Optional<mongoUserDoc> findByUsername(String username);
}

```

B.4 Database Seeders

Listing B.22: sqlSeed.java

```

package github.peaterpita.config;

import java.time.LocalDate;

import java.time.LocalDateTime;

import org.springframework.boot.CommandLineRunner;

```

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

import github.peaterpita.model.*;
import github.peaterpita.repository.*;

@Configuration
@Profile("sql")
public class sqlSeed {

    @Bean
    CommandLineRunner initSqlData(UserRepository userRepo, BookRepository bookRepo, LoanRepository loanRepo) {
        return args -> {

            if (bookRepo.findAll().isEmpty()) {
                // #####
                // # Users #
                // #####
                User USER = new User();
                USER.setUsername("user");
                USER.setPassword("password");
                USER.setCreatedAt(LocalDate.now());

                User SQL = new User();
                SQL.setUsername("sql");
                SQL.setPassword("sql");
                SQL.setCreatedAt(LocalDate.now());

                USER = userRepo.save(USER);
                userRepo.save(SQL);

                // #####
                // # Books #
                // #####
                Book LOTR = new Book();
                LOTR.setIsbn("9780261103573");
                LOTR.setTitle("The Lord of the Rings");
            }
        };
    }
}

```

```
LOTR.setAuthor("J.R.R-Tolkien");  
LOTR.setCopiesLeft(10);
```

```
Book EJ = new Book();  
EJ.setIsbn("9780134685991");  
EJ.setTitle("Effective-Java");  
EJ.setAuthor("Joshua-Bloch");  
EJ.setCopiesLeft(3);
```

```
Book ITENDS = new Book();  
ITENDS.setIsbn("9781501110368");  
ITENDS.setTitle("It-Ends-With-Us");  
ITENDS.setAuthor("Colleen-Hoover");  
ITENDS.setCopiesLeft(0);
```

```
Book TWISTED = new Book();  
TWISTED.setIsbn("9781087939278");  
TWISTED.setTitle("Twisted-Love");  
TWISTED.setAuthor("Ana-Huang");  
TWISTED.setCopiesLeft(2);
```

```
Book IT = new Book();  
IT.setIsbn("9780451159274");  
IT.setTitle("It");  
IT.setAuthor("Stephen-King");  
IT.setCopiesLeft(11);
```

```
Book GOT = new Book();  
GOT.setIsbn("9780007548231");  
GOT.setTitle("A-Game-of-Thrones");  
GOT.setAuthor("George-R.-R.-Martin");  
GOT.setCopiesLeft(3);
```

```
Book HUNGER = new Book();  
HUNGER.setIsbn("9781407135397");  
HUNGER.setTitle("The-Hunger-Games");  
HUNGER.setAuthor("Suzanne-Collins");  
HUNGER.setCopiesLeft(7);
```

```
LOTR = bookRepo.save(LOTR);
EJ = bookRepo.save(EJ);
ITENDS = bookRepo.save(ITENDS);
TWISTED = bookRepo.save(TWISTED);
IT = bookRepo.save(IT);
GOT = bookRepo.save(GOT);
HUNGER = bookRepo.save(HUNGER);
```

```
// #####
// # Loans #
// #####
```

```
String userId = USER.getId();
```

```
Loan loanA = new Loan();
loanA.setUserId(userId);
loanA.setBookId(ITENDS.getId());
loanA.setLoanDate(LocalDate.now().minusDays(21));
loanA.setDueDate(LocalDate.now().minusDays(7));
loanA.setReturnDate(LocalDate.now().minusDays(3));
loanA.setFineAmount(4 * 1.50);
loanA.setStatus("PAID");
```

```
Loan loanB = new Loan();
loanB.setUserId(userId);
loanB.setBookId(IT.getId());
loanB.setLoanDate(LocalDate.now().minusDays(67));
loanB.setDueDate(LocalDate.now().minusDays(53));
loanB.setReturnDate(LocalDate.now().minusDays(53));
loanB.setStatus("RETURNED");
```

```
Loan loanC = new Loan();
loanC.setUserId(userId);
loanC.setBookId(IT.getId());
loanC.setLoanDate(LocalDate.now().minusDays(53));
loanC.setDueDate(LocalDate.now().minusDays(39));
loanC.setReturnDate(LocalDate.now().minusDays(33));
loanC.setFineAmount(6 * 1.50);
```

```

        loanC.setStatus("PAID");

        Loan loanD = new Loan();
        loanD.setUserId(userId);
        loanD.setBookId(GOT.getId());
        loanD.setLoanDate(LocalDate.now().minusDays(3));
        loanD.setDueDate(LocalDate.now().plusDays(11));
        loanD.setStatus("BORROWED");

        Loan loanE = new Loan();
        loanE.setUserId(userId);
        loanE.setBookId(EJ.getId());
        loanE.setLoanDate(LocalDate.now());
        loanE.setDueDate(LocalDate.now().plusDays(14));
        loanE.setStatus("BORROWED");

        Loan loanF = new Loan();
        loanF.setUserId(userId);
        loanF.setBookId(LOTR.getId());
        loanF.setLoanDate(LocalDate.now().minusDays(17));
        loanF.setDueDate(LocalDate.now().minusDays(3));
        loanF.setStatus("BORROWED");
        loanRepo.save(loanA);
        loanRepo.save(loanB);
        loanRepo.save(loanC);
        loanRepo.save(loanD);
        loanRepo.save(loanE);
        loanRepo.save(loanF);
    }
};
}

```

Listing B.23: mongoSeed.java

```

package github.peaterpita.config;

```

```

import github.peaterpita.model.*;
import github.peaterpita.repository.*;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Configuration
@Profile("mongo")
public class mongoSeed {

    // #####
    // # Data population function #
    // # for MongoDB #
    // # Using the same Service and Repository methods used in #
    // # real Use #
    // #####
    @Bean
    CommandLineRunner initMongoData(UserRepository userRepo, BookRepository bookRepo, LoanRepository
        return args -> {
            if (bookRepo.findAll().isEmpty()) {

                User USER = new User();
                USER.setUsername("user");
                USER.setPassword("password");
                USER.setCreatedAt(LocalDate.now());

                User MONGO = new User();
                MONGO.setUsername("mongo");
                MONGO.setPassword("mongo");
                MONGO.setCreatedAt(LocalDate.now());

                USER = userRepo.save(USER);
                userRepo.save(MONGO);

```

```
// #####  
// # Populate Books Document with 10 books #  
// #####
```

```
Book MDB = new Book();  
MDB.setIsbn("978-1491954461");  
MDB.setTitle("MongoDB: -The- Definitive -Guide");  
MDB.setAuthor("Shannon -Bradshaw");  
MDB.setCopiesLeft(10);
```

```
Book CCNA = new Book();  
CCNA.setIsbn("978-0135792735");  
CCNA.setTitle("CCNA-200-301 -Offical -Cert -Guide , -Volume-1");  
CCNA.setAuthor("Wendell -Odom");  
CCNA.setCopiesLeft(3);
```

```
Book FAH = new Book();  
FAH.setIsbn("978-0006546061");  
FAH.setTitle("Fahrenheit -451");  
FAH.setAuthor("Ray -Bradbury");  
FAH.setCopiesLeft(451);
```

```
Book IT = new Book();  
IT.setIsbn("9780451159274");  
IT.setTitle("It");  
IT.setAuthor("Stephen -King");  
IT.setCopiesLeft(11);
```

```
Book DUNE = new Book();  
DUNE.setIsbn("978-0450011849");  
DUNE.setTitle("Dune");  
DUNE.setAuthor("Frank -Herbert");  
DUNE.setCopiesLeft(2);
```

```
Book E101 = new Book();  
E101.setIsbn("9781945796067");  
E101.setTitle("101 -Essays -That -Will -Change -The -Way -You -Think");  
E101.setAuthor("Brianna -Wiest");  
E101.setCopiesLeft(101);
```

```
Book BRIEF = new Book();
BRIEF.setIsbn("0553052438");
BRIEF.setTitle("A-brief-history-of-time");
BRIEF.setAuthor("Stephen-Hawking");
BRIEF.setCopiesLeft(23);
```

```
Book PET = new Book();
PET.setIsbn("9780451150240");
PET.setTitle("Pet-Semetary");
PET.setAuthor("Stephen-King");
PET.setCopiesLeft(5);
```

```
Book MAH = new Book();
MAH.setIsbn("9781449474256");
MAH.setTitle("Milk-and-Honey");
MAH.setAuthor("Rupi-Kaur");
MAH.setCopiesLeft(9);
```

```
Book GUIDE = new Book();
GUIDE.setIsbn("978-0345391803");
GUIDE.setTitle("The-Hitch-Hiker's-Guide-to-the-Galaxy");
GUIDE.setAuthor("Douglas-Adams");
GUIDE.setCopiesLeft(42);
```

```
Book IDIOTS = new Book();
IDIOTS.setIsbn("9781250398956");
IDIOTS.setTitle("Surrounded-by-Idiots");
IDIOTS.setAuthor("Thomas-Erikson");
IDIOTS.setCopiesLeft(0);
```

```
MDB = bookRepo.save(MDB);
CCNA = bookRepo.save(CCNA);
E101 = bookRepo.save(E101);
FAH = bookRepo.save(FAH);
IT = bookRepo.save(IT);
DUNE = bookRepo.save(DUNE);
BRIEF = bookRepo.save(BRIEF);
```

```
MAH = bookRepo.save(MAH);
PET = bookRepo.save(PET);
GUIDE = bookRepo.save(GUIDE);
IDIOTS = bookRepo.save(IDIOTS);
```

```
// #####
// # Loans #
// #####
```

```
String userId = USER.getId();
```

```
Loan loanA = new Loan();
loanA.setUserId(userId);
loanA.setBookId(GUIDE.getId());
loanA.setLoanDate(LocalDate.now().minusDays(23));
loanA.setDueDate(LocalDate.now().minusDays(9));
loanA.setReturnDate(LocalDate.now().minusDays(2));
loanA.setFineAmount(7 * 1.50);
loanA.setStatus("PAID");
```

```
Loan loanB = new Loan();
loanB.setUserId(userId);
loanB.setBookId(IT.getId());
loanB.setLoanDate(LocalDate.now().minusDays(67));
loanB.setDueDate(LocalDate.now().minusDays(53));
loanB.setReturnDate(LocalDate.now().minusDays(53));
loanB.setStatus("RETURNED");
```

```
Loan loanC = new Loan();
loanC.setUserId(userId);
loanC.setBookId(CCNA.getId());
loanC.setLoanDate(LocalDate.now().minusDays(53));
loanC.setDueDate(LocalDate.now().minusDays(39));
loanC.setReturnDate(LocalDate.now().minusDays(33));
loanC.setFineAmount(6 * 1.50);
loanC.setStatus("PAID");
```

```
Loan loanD = new Loan();
loanD.setUserId(userId);
```

```
loanD.setBookId(FAH.getId());
loanD.setLoanDate(LocalDate.now().minusDays(3));
loanD.setDueDate(LocalDate.now().plusDays(11));
loanD.setStatus("BORROWED");
```

```
Loan loanE = new Loan();
loanE.setUserId(userId);
loanE.setBookId(IT.getId());
loanE.setLoanDate(LocalDate.now());
loanE.setDueDate(LocalDate.now().plusDays(14));
loanE.setStatus("BORROWED");
```

```
Loan loanF = new Loan();
loanF.setUserId(userId);
loanF.setBookId(MDB.getId());
loanF.setLoanDate(LocalDate.now().minusDays(17));
loanF.setDueDate(LocalDate.now().minusDays(3));
loanF.setStatus("BORROWED");
```

```
Loan loanG = new Loan();
loanG.setUserId(userId);
loanG.setBookId(GUIDE.getId());
loanG.setLoanDate(LocalDate.now().minusDays(44));
loanG.setDueDate(LocalDate.now().minusDays(30));
loanG.setReturnDate(LocalDate.now().minusDays(33));
loanG.setStatus("RETURNED");
```

```
Loan loanH = new Loan();
loanH.setUserId(userId);
loanH.setBookId(GUIDE.getId());
loanH.setLoanDate(LocalDate.now().minusDays(4));
loanH.setDueDate(LocalDate.now().plusDays(10));
loanH.setStatus("BORROWED");
```

```
loanRepo.save(loanA);
loanRepo.save(loanB);
loanRepo.save(loanC);
loanRepo.save(loanD);
```

```
        loanRepo.save(loanE);
        loanRepo.save(loanF);
        loanRepo.save(loanG);
        loanRepo.save(loanH);
    }
};
}
```

Appendix C

Frontend Implementation

C.1 Core Application

Listing C.1: App.svelte

```
<script>
  import Router, { push } from "svelte-spa-router";
  import { authToken } from "../lib/stores";

  import Login from "../components/Login.svelte";
  import Dashboard from "../components/Dashboard.svelte";
  import Browse from "../components/Browse.svelte";
  import Navbar from "../components/Navbar.svelte";
  import Alert from "../components/Alert.svelte";

  const routes = {
    "/": Dashboard,
    "/login": Login,
    "/browse": Browse,
  };

  $: if (!$authToken) {
    push("/login");
  }
</script>
```

```

<nav>
  {#if $authToken}
    <Navbar />
  {/if}
</nav>

<Alert />
<Router {routes} />

```

Listing C.2: api.js

```

import { authToken } from "../stores";

const API_BASE = 'http://localhost:8082';
let token;

authToken.subscribe(value => {
  token = value;
})

async function request(path, opts = {}) {

  const url = API_BASE + path;
  const headers = opts.headers ? { ...opts.headers } : {};

  if (token) headers['Authorization'] = 'Bearer ' + token;
  headers['Content-Type'] = headers['Content-Type'] || 'application/json';

  const res = await fetch(url, { ...opts, headers });
  if (res.status === 401) {
    authToken.set(null);
    throw new Error("Unauthorized");
  }

  let data;
  const contentType = res.headers.get('content-type') || '';
  if (contentType.includes('application/json')) {

```

```

        data = await res.json();
    } else {
        data = await res.text();
    }

    if (!res.ok) {
        const err = (typeof data === 'object' && data.error) ? data.error : (data || res.statusText);
        throw new Error(err);
    }

    return data;
}

```

```

export async function login(username, password) {
    const res = await fetch(APIBASE + '/api/auth/login', {
        method: 'POST',
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username, password })
    });
    if (!res.ok) {
        const err = await res.json().catch(() => ({ message: 'Login-failed' }));
        throw new Error(err.message || 'bad');
    }
    const body = await res.json();
    return body.token;
}

```

```

export async function fetchBooks() {
    return request('/api/books');
}

```

```

export async function searchBooks(query = '', available = false) {
    const urlBuild = new URLSearchParams();
    if (query && query.trim().length > 0) urlBuild.append("query", query.trim());
    if (available) urlBuild.append('available', 'true');
}

```

```

    const path = '/api/books' + (urlBuild.toString() ? ('?' + urlBuild.toString()) : '');
    return request(path);
  }

  export async function fetchBookById(id) {
    return request(`/api/books/${id}`);
  }


  export async function fetchLoansForUser() {
    return request('/api/loans/me');

  }


  export async function borrowBook(bookId) {
    return request('/api/loans/borrow', {
      method: 'POST',
      body: JSON.stringify({ bookId })
    });
  }


  export async function returnBook(loanId) {
    return request('/api/loans/return', {
      method: 'POST',
      body: JSON.stringify({ loanId })
    });
  }


  export async function payFine(loanId) {
    return request(`/api/loans/pay/${loanId}`, { method: 'POST' });
  }
}

```

Listing C.3: stores.js

```

import { derived, writable } from "svelte/store";

```

```

export const authToken = writable(localStorage.getItem('jwt-token'));
authToken.subscribe(value => {
  if (value) localStorage.setItem('jwt-token', value);
  else localStorage.removeItem('jwt-token')
});

export const alerts = writable([]);

export function addAlert(message, style) {
  const id = Math.random();

  alerts.update((prev) => [
    ...prev,
    { id, message, style }
  ]);

  setTimeout(() => {
    removeAlert(id);
  }, 5000);
}

function removeAlert(id) {
  alerts.update((prev) => prev.filter((alert) => alert.id !== id));
}

export const currentUser = derived(authToken, ($auth) => {
  try {
    const payload = JSON.parse(atob($auth.split('.')[1]));
    return payload.sub || "ERROR";
  } catch (err) {
    console.error("Error with the token username fetch");
    return null;
  }
});

```

C.2 Components

Listing C.4: Dashboard.svelte

```
<script>
import { push } from "svelte-spa-router";
import { fetchLoansForUser, payFine, returnBook } from "../lib/api";
import { onMount } from "svelte";
import BookCard from "./BookCard.svelte";
import LoanPanel from "./loanPanel.svelte";
import { addAlert } from "../lib/stores";

let loans = [];
let loading = true;

async function handlePay(loanId, fineAmount) {
  if (!confirm('Are you sure?\nPayment Due: £${fineAmount}')) return;
  try {
    await payFine(loanId);
    addAlert('Payment of £${fineAmount} complete!', "success");
    await load();
  } catch (err) {
    addAlert(err.message);
  }
}

async function loanReturn(loanId) {
  try {
    await returnBook(loanId);
    addAlert('Book returned', "success");
    await load();
  } catch (err) {
    addAlert(err.message);
  }
}

async function load() {
  try {
    loans = await fetchLoansForUser();
  } catch (err) {
    addAlert(err.message);
  }
}
```

```

        if (err.message === "Unauthorized") {
            push("/login");
        }
    } finally {
        loading = false;
    }
}

onMount(() => {
    load();
});

$: currentLoans = loans.filter((loan) => !loan.returnDate);
$: finesList = loans.filter((loan) => loan.status === "OVERDUE");
</script>

<div>
    {#if loading}
        <p>Loading... </p>
    { :else }
        <section>
            <h3>Reading List </h3>
            {#if currentLoans.length === 0}
                <p>No books currently on loan!</p>
            { :else }
                <div class="grid">
                    {#each currentLoans as loan}
                        <BookCard
                            book={{ ...loan.book, dueDate: loan.dueDate }}
                            showDue={true}
                        />
                    {/each}
                </div>
            {/if}
        </section>

    <hr />

```



```

        </div>
      </div>
    { :else }
  </div>
  <p>No fines need to be paid</p>
</div>
{ /each }
</div>
</div>
</div>
{ /if }
</div>

```

Listing C.5: BookCard.svelte

```

<script>
  export let book;
  export let showActions = false;
  export let showDue = false;
  export let handleLoan = (-1, -2) => {};
</script>

<!-- svelte-ignore ally_click_events_have_key_events -->
<!-- svelte-ignore ally_no_static_element_interactions -->
<div class="active-card">
  <img src={book.coverURL} alt="Cover" />
  <div class="book-meta">
    <div class="book-title">{book.title}</div>
    <div class="book-author">{book.author}</div>

    { #if showActions }
      <div class="book-copies">
        {book.copiesLeft
          ? 'Copies: ${book.copiesLeft}'
          : "Out of Stock"}
      </div>

      <button
        class="loanBtn"

```

```

        disabled={book.copiesLeft <= 0}
        on:click={() => handleLoan(book.id, book.title)}>Loan</button>
    >
    { :else if showDue }
    <span class="label">Return by: {book.dueDate}</span>
  { /if }
</div>
</div>

```

Listing C.6: Browse.svelte

```

<script>
  import { borrowBook, searchBooks } from "../lib/api";
  import { onMount } from "svelte";
  import BookCard from "../BookCard.svelte";
  import { addAlert } from "../lib/stores";

  const DEBOUNCE_TIME_MS = 500;

  let availableOnly = false;
  let books = [];
  let loading = true;
  let debounceTimer = null;
  let query = "";

  async function load() {
    loading = true;
    try {
      books = await searchBooks(query, availableOnly);
    } catch (err) {
      addAlert(err.message);
    } finally {
      loading = false;
    }
  }

  onMount(load);

  function onQueryChange(i) {

```

```

        query = i.target.value;
        if (debounceTimer) clearTimeout(debounceTimer);
        debounceTimer = setTimeout(() => load(), DEBOUNCE_TIME_MS);
    }

    async function handleLoanOut(bookId, bookTitle) {
        try {
            await borrowBook(bookId);
            addAlert('Book borrowed: ${bookTitle}', "success");
            await load();
        } catch (err) {
            addAlert(err.message, "error");
        }
    }
</script>

<div>
    <h2>Browse Books</h2>

    <div class="controls">
        <input
            placeholder="Search by title or author ..."
            value={query}
            on:input={onQueryChange}
        />
        <label
            ><input
                type="checkbox"
                bind:checked={availableOnly}
                on:change={load}
            /> Available only</label
        >
    </div>

    {#if loading}
        <p class="center">Loading books... </p>
    { :else if books.length === 0}
        <p class="center">No books found</p>
    { /if }

```

```

    { :else }
    <div class="grid">
      {#each books as book}
        <BookCard
          {book}
          showActions={true}
          handleLoan={handleLoanOut}
        />
      {/each}
    </div>
  {/if}
</div>

<style>
  .controls {
    display: flex;
    gap: 10px;
    align-items: center;
    margin-bottom: 1rem;
  }
  .center {
    text-align: center;
    color: #666;
  }
</style>

```

Listing C.7: Login.svelte

```

<script>
  import { push } from "svelte-spa-router";
  import { login } from "../lib/api";
  import { authToken } from "../lib/stores";

  let username = "user";
  let password = "password";
  let error = "";
  let loading = false;

  async function onSubmit(event) {

```

```

    event.preventDefault();
    error = "";
    loading = true;
    try {
        const token = await login(username, password);

        authToken.set(token);

        push("/");
    } catch (err) {
        error = err.message || "Failed to login. Please check creds";
    } finally {
        loading = false;
    }
}
</script>

<div class="panel-login-form">
    <h2>Login</h2>
    <form on:submit={onSubmit}>
        <label> Username <input bind:value={username} /> </label>

        <label> Password <input bind:value={password} type="password" /></label>

        <div style="margin-top:10px;">
            <button type="submit" disabled={loading}
                >{loading ? "Logging in ..." : "Login"}</button>
        >
    </div>

    {#if error}
        <p style="color:red;">{error}</p>
    {/if}
</form>
</div>

```

Listing C.8: Navbar.svelte

```

<script>

```

```

import { link } from "svelte-spa-router";
import { authToken, currentUser } from "../lib/stores";

function logout() {
    authToken.set(null);
}
</script>

<nav class="navbar">
  <div class="container nav-container">
    <a use:link href="/" class="nav-brand">DBA1<span>Library</span></a>
    <div class="nav-links">
      <a use:link href="/browse" class="nav-item">Browse</a>
      <div class="ctaMenu">
        <button on:click={logout} class="nav-logout">Logout</button>
        {#if currentUser}
          <div class="label">
            Logged in as: <strong>{currentUser}</strong>
          </div>
        {/if}
      </div>
    </div>
  </div>
</nav>

```

Listing C.9: loanPanel.svelte

```

<script>
  export let loans = [];
  export let onReturn;

  let startDate = "";
  let endDate = "";

  $: loansHistory = loans
    .filter((loan) => {
      if (!startDate && !endDate) return true;

      const loanDate = new Date(loan.loanDate);

```

```

        const start = startDate
        ? new Date(startDate)
        : new Date("2010-01-01");
        const end = endDate ? new Date(endDate) : new Date(Date.now());

        return loanDate >= start && loanDate <= end;
    })
    .sort((l1, l2) => {
        return (
            new Date(l2.loanDate).getTime() -
            new Date(l1.loanDate).getTime()
        );
    });

$: totalPaid = loansHistory
    .filter((loan) => loan.status === "PAID")
    .reduce((sum, loan) => sum + (loan.fine || 0), 0);

function genTags(loan) {
    let tagList = [];

    if (loan.returnDate) {
        tagList.push({ text: "Returned", style: "returned" });

        if (new Date(loan.returnDate) > new Date(loan.dueDate)) {
            tagList.push({ text: "Late", style: "late" });
        }
    } else if (loan.status === "OVERDUE") {
        tagList.push({ text: "Overdue", style: "late" });
    } else {
        tagList.push({ text: "Borrowed", style: "active" });
    }
    return tagList;
}
</script>

<div class="panel-history-panel">
    <div class="panel-header">

```

```

<div class="header-left">
  <h2>Loan History</h2>
  <div class="total-badge">
    {#if totalPaid > 0}
      <span>Total paid: £{totalPaid.toFixed(2)}</span>
    {/if}
  <span>
    >Late returns: {loansHistory.filter(
      (loan) => loan.status === "PAID",
    ).length}</span>
  </div>
</div>
<div class="date-controls">
  <label>
    <span>From:</span>
    <input type="date" bind:value={startDate} />
  </label>
  <label>
    <span>To:</span>
    <input type="date" bind:value={endDate} />
  </label>
</div>
</div>

<div class="table-container">
  <table>
    <thead>
      <tr>
        <th>Book (Total: {loansHistory.length})</th>
        <th>Loaned</th>
        <th>Due</th>
        <th>Status</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody>
      {#each loansHistory as loan}

```

```

<tr>
  <td>{loan.book ? loan.book.title : "Error"}</td>
  <td>{loan.loanDate}</td>
  <td>{loan.dueDate}</td>
  <td>
    <div class="tagRow">
      {#each genTags(loan) as tag}
        <span class="tag-{tag.style}"
          >{tag.text}</span>
      >
    {/each}
  </div>
</td>
<td>
  {#if !loan.returnDate && loan.status != "OVERDUE"}
    <button
      class="btn"
      on:click={() => onReturn(loan.id)}
      >Return</button>
  >
  { :else if loan.status == "PAID" }
    <span>£{loan.fine.toFixed(2)} Paid</span>
  { :else if !loan.returnDate && loan.status == "OVERDUE" }
    <span class="tag-late">Pay Fine to Return</span>
  { :else }{ /if }
</td>
</tr>
{ :else }
<tr>
  ><td colspan="5" style="text-align:center">
    >No history available</td>
  ></tr>
>
{/each}
</tbody>
</table>
</div>
</div>

```